Python笔试题(4)

🚀 算子测试平台 - 完整知识点覆盖项目

我为你设计了一个**算子测试平台**项目，自然融合了你整理的所有知识点。这个项目模拟真实的AI算子测试场景。

📁 项目结构

```
operator_test_platform/
├── src/
│   ├── __init__.py
│   ├── operators/
│   │   ├── __init__.py
│   │   ├── base.py          # 知识点：元类、描述符、类型注解
│   │   ├── matmul.py        # 知识点：多线程、深浅拷贝
│   │   ├── conv2d.py        # 知识点：async/await、上下文管理器
│   │   └── pooling.py       # 知识点：生成器、迭代器
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── decorators.py    # 知识点：装饰器、闭包、wraps
│   │   ├── data_gen.py      # 知识点：yield、迭代器协议
│   │   ├── memory.py        # 知识点：gc模块、引用计数
│   │   ├── context.py       # 知识点：上下文管理器
│   │   └── performance.py   # 知识点：timeit、性能优化
│   └── config/
│       ├── __init__.py
│       ├── loader.py        # 知识点：TypedDict、Generic
│       └── settings.yaml
├── tests/
│   ├── conftest.py          # 知识点：fixture scope、autouse
│   ├── unit/
│   │   ├── conftest.py      # 知识点：fixture层级查找
│   │   └── test_operators.py # 知识点：parametrize、mark
│   ├── integration/
│   │   └── test_pipeline.py  # 知识点：多线程测试
│   └── performance/
│       └── test_benchmark.py # 知识点：Allure报告
├── pytest.ini
├── requirements.txt
└── README.md
```

💻 完整代码实现

1. src/operators/base.py (元类、描述符、property)

```
"""
算子基类模块
知识点：元类、描述符、property、类型注解
"""
from typing import TypeVar, Generic, Optional, Type
from abc import ABCMeta, abstractmethod
import numpy as np

T = TypeVar('T', bound=np.ndarray)


class ShapeValidator:
    """
    描述符：验证shape有效性
    知识点：描述符协议 __get__/__set__/__delete__
    """
    def __init__(self, name: str):
        self.name = f"_{name}"

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return getattr(instance, self.name, None)

    def __set__(self, instance, value):
        if not isinstance(value, (tuple, list)):
            raise TypeError(f"Shape必须是tuple或list，得到{type(value)}")
        if any(dim <= 0 for dim in value):
            raise ValueError(f"Shape维度必须>0，得到{value}")
        setattr(instance, self.name, tuple(value))

    def __delete__(self, instance):
        """知识点：del操作符触发描述符的__delete__"""
        print(f"[DEBUG] 删除{self.name}属性")
        delattr(instance, self.name)


class OperatorMeta(ABCMeta):
    """
    算子元类
    知识点：元类、自动注册机制
    """
    _registry = {}
```

```
    def __new__(mcs, name, bases, attrs):
        cls = super().__new__(mcs, name, bases, attrs)
        # 自动注册算子类（非基类）
        if name != 'BaseOperator':
            mcs._registry[name] = cls
            print(f"[META] 注册算子: {name}")
        return cls

    @classmethod
    def get_operator(mcs, name: str) -> Optional[Type]:
        """通过名字获取算子类"""
        return mcs._registry.get(name)


class BaseOperator(Generic[T], metaclass=OperatorMeta):
    """
    算子基类
    知识点：元类、泛型、抽象方法、property
    """

    # 使用描述符
    input_shape = ShapeValidator("input_shape")
    output_shape = ShapeValidator("output_shape")

    def __init__(self, name: str, device: str = 'cpu'):
        self._name = name
        self._device = device
        self._call_count = 0

    @property
    def name(self) -> str:
        """知识点：property作为只读属性"""
        return self._name

    @property
    def call_count(self) -> int:
        """调用计数（演示property的get）"""
        return self._call_count

    @call_count.setter
    def call_count(self, value: int):
        """知识点：property的setter"""
        if value < 0:
            raise ValueError("call_count不能为负")
        self._call_count = value

    @abstractmethod
```

```
    def forward(self, *inputs: T) -> T:
        """抽象方法：子类必须实现"""
        pass

    def __call__(self, *inputs: T) -> T:
        """使实例可调用"""
        self._call_count += 1
        return self.forward(*inputs)
```

2. src/utils/decorators.py (装饰器、闭包、wraps)

```
"""
装饰器工具模块
知识点：带参数的装饰器、闭包、functools.wraps、类型注解
"""
import functools
import time
import threading
from typing import Callable, Any, TypeVar, Optional

F = TypeVar('F', bound=Callable[..., Any])


def timer(unit: str = 'ms'):
    """
    计时装饰器（带参数）
    知识点：三层闭包、装饰器参数、functools.wraps
    """
    def decorator(func: F) -> F:
        @functools.wraps(func)  # 知识点：保留原函数元信息
        def wrapper(*args, **kwargs):
            start = time.perf_counter()
            result = func(*args, **kwargs)
            elapsed = time.perf_counter() - start

            if unit == 'ms':
                elapsed *= 1000
            elif unit == 's':
                pass
            else:
                raise ValueError(f"不支持的单位: {unit}")

            print(f"[TIMER] {func.__name__} 耗时: {elapsed:.4f}{unit}")
            return result
        return wrapper
    return decorator
```

```python
def retry(max_attempts: int = 3, delay: float = 0.1):
    """
    重试装饰器
    知识点：装饰器实现重试逻辑
    """
    def decorator(func: F) -> F:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts:
                        raise
                    print(f"[RETRY] 第{attempt}次失败: {e}, {delay}s后重试...")
                    time.sleep(delay)
        return wrapper
    return decorator


def thread_safe(func: F) -> F:
    """
    线程安全装饰器
    知识点：装饰器中使用Lock、闭包保存状态
    """
    lock = threading.Lock()  # 闭包变量

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        with lock:  # 知识点：上下文管理器
            return func(*args, **kwargs)

    # 暴露lock供外部查看
    wrapper.lock = lock
    return wrapper


class CountCalls:
    """
    基于类的装饰器
    知识点：__call__实现装饰器、实例属性
    """
    def __init__(self, func: Callable):
        functools.update_wrapper(self, func)
        self.func = func
        self.count = 0
```

```
    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"[COUNT] {self.func.__name__} 被调用 {self.count} 次")
        return self.func(*args, **kwargs)
```

3. src/utils/data_gen.py (生成器、迭代器、yield)

```
"""
数据生成器模块
知识点：生成器、yield、迭代器协议、iter()/next()
"""
import numpy as np
from typing import Iterator, Tuple, Generator
from collections.abc import Iterable


class TensorGenerator:
    """
    自定义迭代器：生成测试张量
    知识点：实现迭代器协议 __iter__/__next__
    """
    def __init__(self, shapes: list, max_count: int = 10):
        self.shapes = shapes
        self.max_count = max_count
        self.current = 0

    def __iter__(self):
        """知识点：__iter__返回自身"""
        return self

    def __next__(self) -> np.ndarray:
        """知识点：__next__返回下一个元素，无元素时抛StopIteration"""
        if self.current >= self.max_count:
            raise StopIteration

        shape = self.shapes[self.current % len(self.shapes)]
        self.current += 1
        return np.random.randn(*shape).astype(np.float32)


def batch_data_generator(data: np.ndarray, batch_size: int) ->
Generator[np.ndarray, None, None]:
    """
    批量数据生成器
    知识点：yield生成器、惰性求值
```

```
    """
    total = len(data)
    for start in range(0, total, batch_size):
        end = min(start + batch_size, total)
        print(f"[GEN] 生成batch [{start}:{end}]")
        yield data[start:end]  # 知识点: yield暂停并返回值


def infinite_data_stream(shape: Tuple[int, ...]) -> Generator[np.ndarray, None,
None]:
    """
    无限数据流生成器
    知识点: yield实现无限生成器
    """
    count = 0
    while True:
        count += 1
        print(f"[STREAM] 生成第{count}个数据")
        yield np.random.randn(*shape)


def send_generator_example():
    """
    演示生成器的send方法
    知识点: yield作为表达式、send()双向通信
    """
    def echo_generator():
        value = None
        while True:
            received = yield value  # 知识点: yield接收send的值
            if received is not None:
                value = f"Echo: {received}"

    gen = echo_generator()
    next(gen)  # 初始化生成器
    print(gen.send("Hello"))  # 输出: Echo: Hello
    print(gen.send("World"))  # 输出: Echo: World
```

4. src/operators/matmul.py (多线程、深浅拷贝)

```
"""
矩阵乘法算子
知识点: threading、multiprocessing、深浅拷贝
"""
import numpy as np
import threading
```

```python
import multiprocessing as mp
import copy
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
from typing import List
from .base import BaseOperator
from ..utils.decorators import timer, thread_safe


class MatMulOperator(BaseOperator):
    """矩阵乘法算子"""

    def __init__(self, name: str = "MatMul", use_cache: bool = True):
        super().__init__(name)
        self._cache = {}  # 知识点：浅拷贝会共享这个字典
        self.use_cache = use_cache

    @timer(unit='ms')
    def forward(self, A: np.ndarray, B: np.ndarray) -> np.ndarray:
        """前向计算"""
        return np.matmul(A, B)

    @thread_safe
    def cached_forward(self, A: np.ndarray, B: np.ndarray) -> np.ndarray:
        """
        带缓存的计算（线程安全）
        知识点：装饰器实现线程安全
        """
        key = (A.shape, B.shape, id(A), id(B))
        if self.use_cache and key in self._cache:
            print(f"[CACHE] 命中缓存")
            return self._cache[key]

        result = self.forward(A, B)
        self._cache[key] = result
        return result

    def __deepcopy__(self, memo):
        """
        自定义深拷贝
        知识点：深拷贝会递归复制_cache
        """
        print("[COPY] 执行深拷贝")
        new_obj = type(self)(self.name, self.use_cache)
        new_obj._cache = copy.deepcopy(self._cache, memo)
        return new_obj


def parallel_matmul_threading(matrices: List[np.ndarray], B: np.ndarray,
```

```
max_workers: int = 4):
    """
    多线程并行矩阵乘法
    知识点：ThreadPoolExecutor、GIL影响
    """
    op = MatMulOperator()

    with ThreadPoolExecutor(max_workers=max_workers) as executor:
        # 知识点：使用map进行并行计算
        results = list(executor.map(lambda A: op.forward(A, B), matrices))

    return results


def parallel_matmul_multiprocessing(matrices: List[np.ndarray], B: np.ndarray,
max_workers: int = 4):
    """
    多进程并行矩阵乘法
    知识点：ProcessPoolExecutor绕过GIL、进程间通信
    """
    def compute(A):
        op = MatMulOperator()
        return op.forward(A, B)

    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        results = list(executor.map(compute, matrices))

    return results
```

5. src/operators/conv2d.py (async/await、上下文管理器)

```
"""
卷积算子
知识点：asyncio、协程、上下文管理器
"""
import asyncio
import numpy as np
from contextlib import contextmanager
from .base import BaseOperator
from ..utils.decorators import timer


class Conv2dOperator(BaseOperator):
    """卷积算子"""

    def __init__(self, in_channels: int, out_channels: int, kernel_size: int):
```

```
        super().__init__("Conv2d")
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.kernel_size = kernel_size
        self.weight = np.random.randn(out_channels, in_channels, kernel_size,
kernel_size)

    def forward(self, x: np.ndarray) -> np.ndarray:
        """简化的卷积实现"""
        # 这里用简单的逻辑模拟，实际应该是真正的卷积运算
        return np.random.randn(x.shape[0], self.out_channels, x.shape[2]//2,
x.shape[3]//2)

    @contextmanager
    def training_mode(self):
        """
        上下文管理器：训练模式
        知识点：@contextmanager装饰器、yield实现上下文
        """
        print("[CTX] 进入训练模式")
        old_weight = self.weight.copy()
        try:
            yield self  # 知识点：yield把控制权交给with块
        except Exception as e:
            print(f"[CTX] 异常发生，恢复权重：{e}")
            self.weight = old_weight
            raise
        finally:
            print("[CTX] 退出训练模式")

    async def async_forward(self, x: np.ndarray) -> np.ndarray:
        """
        异步前向计算
        知识点：async def定义协程、await暂停
        """
        print(f"[ASYNC] 开始异步计算 (shape={x.shape})")
        await asyncio.sleep(0.1)  # 模拟I/O等待
        result = self.forward(x)
        print(f"[ASYNC] 完成异步计算")
        return result


async def batch_async_inference(op: Conv2dOperator, inputs: list):
    """
    批量异步推理
    知识点：asyncio.gather并发执行多个协程
    """
    tasks = [op.async_forward(x) for x in inputs]
```

```
    results = await asyncio.gather(*tasks)  # 知识点：gather并发等待
    return results
```

6. src/utils/memory.py (gc模块、引用计数、del)

```
"""
内存管理模块
知识点：gc模块、引用计数、弱引用、del操作
"""
import gc
import sys
import weakref
import numpy as np
from typing import List


class MemoryTracker:
    """内存跟踪器"""

    def __init__(self, name: str):
        self.name = name
        self.data = np.random.randn(1000, 1000)  # 占用内存
        print(f"[MEM] {name} 创建，引用计数={sys.getrefcount(self)}")

    def __del__(self):
        """
        析构函数
        知识点：__del__在对象被回收时调用
        """
        print(f"[MEM] {self.name} 被回收")


def demo_reference_counting():
    """
    演示引用计数
    知识点：sys.getrefcount、del触发回收
    """
    obj = MemoryTracker("obj1")
    print(f"引用计数: {sys.getrefcount(obj)}")  # 2 (obj + getrefcount的参数)

    ref1 = obj
    print(f"引用计数: {sys.getrefcount(obj)}")  # 3

    del ref1  # 知识点：del减少引用计数
    print(f"引用计数: {sys.getrefcount(obj)}")  # 2
```

```
    del obj  # 知识点：引用计数归0，触发__del__


def demo_circular_reference():
    """
    演示循环引用
    知识点：循环引用不会被引用计数回收，需要gc
    """
    class Node:
        def __init__(self, name):
            self.name = name
            self.ref = None

        def __del__(self):
            print(f"[GC] Node {self.name} 被回收")

    a = Node("A")
    b = Node("B")
    a.ref = b  # A引用B
    b.ref = a  # B引用A，形成循环

    del a
    del b
    print("[GC] 删除a和b后，它们不会立即回收（循环引用）")

    gc.collect()  # 知识点：手动触发垃圾回收
    print("[GC] gc.collect()后，循环引用被打破")


def demo_weak_reference():
    """
    演示弱引用
    知识点：weakref不增加引用计数
    """
    obj = MemoryTracker("weak_obj")
    weak_obj = weakref.ref(obj)  # 知识点：创建弱引用

    print(f"弱引用是否存活：{weak_obj() is not None}")
    print(f"引用计数：{sys.getrefcount(obj)}")  # 弱引用不增加计数

    del obj
    print(f"del后弱引用：{weak_obj()}")  # None
```

7. src/config/loader.py (TypedDict、Generic、TypeVar)

```python
"""
配置加载模块
知识点：类型注解、TypedDict、Generic、mypy
"""
from typing import TypedDict, List, Dict, Any, Optional, Union, Generic, TypeVar
from typing_extensions import NotRequired  # Python 3.11+
import yaml


class OperatorConfig(TypedDict):
    """
    算子配置类型
    知识点：TypedDict提供结构化字典类型
    """
    name: str
    device: str
    batch_size: int
    input_shape: List[int]
    precision: NotRequired[str]  # 知识点：可选字段


class TestConfig(TypedDict):
    """测试配置"""
    operators: List[OperatorConfig]
    parallel: bool
    max_workers: int


T = TypeVar('T', bound=Dict[str, Any])


class ConfigLoader(Generic[T]):
    """
    泛型配置加载器
    知识点：Generic实现类型参数化
    """

    def __init__(self, config_type: type):
        self.config_type = config_type

    def load(self, path: str) -> T:
        """加载YAML配置"""
        with open(path, 'r', encoding='utf-8') as f:
            data = yaml.safe_load(f)
        return self._validate(data)
```

```
    def _validate(self, data: Dict[str, Any]) -> T:
        """
        验证配置类型
        知识点：运行时类型检查
        """
        # 实际项目中应使用pydantic或类似库
        return data  # type: ignore


def type_annotation_examples():
    """类型注解示例"""

    # Union类型
    def process(value: Union[int, str]) -> str:
        return str(value)

    # Optional等价于Union[T, None]
    def find_operator(name: str) -> Optional[OperatorConfig]:
        return None

    # 复杂嵌套类型
    cache: Dict[str, List[Optional[np.ndarray]]] = {}
```

8. tests/conftest.py (Fixture、Scope、Autouse)

```
"""
全局conftest配置
知识点：fixture scope、autouse、session/module/class/function级别
"""
import pytest
import numpy as np
from typing import Generator
import sys
from pathlib import Path

# 知识点：添加src到路径
sys.path.insert(0, str(Path(__file__).parent.parent / 'src'))

from operators.matmul import MatMulOperator
from operators.conv2d import Conv2dOperator


@pytest.fixture(scope="session", autouse=True)
def setup_test_environment():
    """
    Session级fixture，自动使用
```

```
    知识点: scope="session"在整个测试会话中只执行一次
    知识点: autouse=True自动应用到所有测试
    """
    print("\n[SESSION] ========== 测试会话开始 ==========")
    np.random.seed(42)  # 固定随机种子
    yield
    print("\n[SESSION] ========== 测试会话结束 ==========")


@pytest.fixture(scope="module")
def shared_matmul_op():
    """
    Module级fixture
    知识点: 同一module内的测试共享该fixture
    """
    print("\n[MODULE] 创建共享的MatMul算子")
    op = MatMulOperator(name="SharedMatMul")
    yield op
    print("\n[MODULE] 清理共享的MatMul算子")


@pytest.fixture(scope="class")
def class_level_data():
    """
    Class级fixture
    知识点: 同一测试类内共享
    """
    print("\n[CLASS] 生成类级别测试数据")
    data = {
        'A': np.random.randn(10, 10),
        'B': np.random.randn(10, 10)
    }
    yield data


@pytest.fixture(scope="function")
def test_matrices():
    """
    Function级fixture (默认)
    知识点: 每个测试函数都会重新执行
    """
    print("\n[FUNCTION] 生成测试矩阵")
    A = np.random.randn(4, 4).astype(np.float32)
    B = np.random.randn(4, 4).astype(np.float32)
    return A, B


@pytest.fixture
```

```python
def conv_op():
    """卷积算子fixture"""
    return Conv2dOperator(in_channels=3, out_channels=64, kernel_size=3)


@pytest.fixture(params=[2, 4, 8])
def batch_size(request):
    """
    参数化fixture
    知识点：fixture也可以参数化
    """
    return request.param


# 知识点：环境清理fixture
@pytest.fixture
def gpu_memory_guard():
    """GPU内存保护"""
    import gc
    gc.collect()  # 测试前清理
    yield
    gc.collect()  # 测试后清理
```

9. tests/unit/conftest.py (Fixture层级查找)

```python
"""
单元测试级conftest
知识点：conftest.py层级查找规则
- pytest会从测试文件所在目录向上查找conftest.py
- 子目录的conftest可以覆盖父目录的同名fixture
"""
import pytest
import numpy as np


@pytest.fixture
def small_matrices():
    """
    单元测试专用的小矩阵
    知识点：同名fixture覆盖上级conftest
    """
    print("\n[UNIT] 生成小矩阵 (2x2)")
    A = np.array([[1, 2], [3, 4]], dtype=np.float32)
    B = np.array([[5, 6], [7, 8]], dtype=np.float32)
    return A, B
```

```python
@pytest.fixture
def mock_operator():
    """Mock算子（仅单元测试可用）"""
    from operators.base import BaseOperator

    class MockOperator(BaseOperator):
        def forward(self, *inputs):
            return inputs[0] * 2

    return MockOperator(name="Mock")
```

10. tests/unit/test_operators.py (Parametrize、Mark、Assert)

```python
"""
算子单元测试
知识点：parametrize、mark、assert高级用法
"""
import pytest
import numpy as np
from operators.matmul import MatMulOperator, parallel_matmul_threading
from operators.conv2d import Conv2dOperator
from utils.decorators import CountCalls
import sys


class TestMatMulOperator:
    """测试矩阵乘法算子（class组织测试）"""

    @pytest.mark.parametrize("shape_a,shape_b,expected_shape", [
        ((2, 3), (3, 4), (2, 4)),
        ((5, 10), (10, 20), (5, 20)),
        ((1, 100), (100, 1), (1, 1)),
    ], ids=["small", "medium", "vector"])  # 知识点：ids自定义测试用例名称
    def test_output_shape(self, shape_a, shape_b, expected_shape):
        """
        参数化测试输出shape
        知识点：@pytest.mark.parametrize多参数
        """
        op = MatMulOperator()
        A = np.random.randn(*shape_a).astype(np.float32)
        B = np.random.randn(*shape_b).astype(np.float32)

        result = op.forward(A, B)

        assert result.shape == expected_shape  # 知识点：pytest自动增强assert
```

```python
    @pytest.mark.parametrize("dtype", [np.float16, np.float32, np.float64])
    def test_dtype_support(self, dtype):
        """测试不同数据类型"""
        op = MatMulOperator()
        A = np.random.randn(3, 3).astype(dtype)
        B = np.random.randn(3, 3).astype(dtype)

        result = op.forward(A, B)
        assert result.dtype == dtype

    @pytest.mark.smoke  # 知识点：自定义mark标记
    def test_basic_correctness(self, test_matrices):
        """冒烟测试：基本正确性"""
        op = MatMulOperator()
        A, B = test_matrices

        result = op.forward(A, B)
        expected = np.matmul(A, B)

        # 知识点：numpy数组的assert
        np.testing.assert_allclose(result, expected, rtol=1e-5)

    @pytest.mark.slow  # 知识点：标记慢速测试
    def test_large_matrix(self):
        """大矩阵测试"""
        op = MatMulOperator()
        A = np.random.randn(1000, 1000).astype(np.float32)
        B = np.random.randn(1000, 1000).astype(np.float32)

        result = op.forward(A, B)
        assert result.shape == (1000, 1000)

    @pytest.mark.skipif(sys.platform == "win32", reason="Windows不支持")
    def test_platform_specific(self):
        """知识点：条件跳过测试"""
        pass

    def test_cache_mechanism(self, test_matrices):
        """测试缓存机制"""
        op = MatMulOperator(use_cache=True)
        A, B = test_matrices

        # 第一次调用
        result1 = op.cached_forward(A, B)
        cache_size_1 = len(op._cache)

        # 第二次调用（应该命中缓存）
```

```
        result2 = op.cached_forward(A, B)
        cache_size_2 = len(op._cache)

        assert cache_size_1 == cache_size_2  # 缓存数量不变
        np.testing.assert_array_equal(result1, result2)


@pytest.mark.threading  # 知识点: 自定义mark组织测试
class TestThreading:
    """多线程测试"""

    def test_parallel_matmul(self):
        """测试多线程矩阵乘法"""
        matrices = [np.random.randn(10, 10).astype(np.float32) for _ in
range(8)]
        B = np.random.randn(10, 10).astype(np.float32)

        results = parallel_matmul_threading(matrices, B, max_workers=4)

        assert len(results) == 8
        for result in results:
            assert result.shape == (10, 10)


# 知识点: pytest-assume 测试多个条件
pytest_plugins = ['pytest_assume']

def test_multiple_assertions_assume():
    """
    使用pytest-assume测试多个条件
    知识点: 普通assert第一个失败就停止, assume会全部执行
    """
    op = MatMulOperator()

    pytest.assume(op.name == "MatMul")
    pytest.assume(op.call_count == 0)
    pytest.assume(op._device == 'cpu')


# 知识点: dict比对
def test_dict_comparison():
    """字典比对技巧"""
    result = {'acc': 0.95, 'loss': 0.05}
    expected = {'acc': 0.95, 'loss': 0.05}

    # 知识点: pytest会展示dict差异
    assert result == expected
```

```
    # 更详细的dict比对
    assert result.keys() == expected.keys()
    for key in result:
        assert abs(result[key] - expected[key]) < 1e-6
```

11. tests/performance/test_benchmark.py (Allure报告、性能测试)

```
"""
性能基准测试
知识点: Allure报告、allure.step、allure.attach
"""
import pytest
import allure
import numpy as np
from operators.matmul import MatMulOperator
from utils.performance import benchmark
import time


@allure.feature('性能测试')  # 知识点: Allure的feature层级
@allure.story('矩阵乘法性能')  # 知识点: story子层级
class TestPerformance:

    @allure.title("小矩阵性能测试 - {size}x{size}")
    @pytest.mark.parametrize("size", [10, 50, 100, 500])
    def test_small_matrix_performance(self, size):
        """
        知识点: Allure标题参数化
        """
        op = MatMulOperator()

        with allure.step(f"生成{size}x{size}测试矩阵"):
            A = np.random.randn(size, size).astype(np.float32)
            B = np.random.randn(size, size).astype(np.float32)

        with allure.step("执行矩阵乘法并计时"):
            start = time.perf_counter()
            result = op.forward(A, B)
            elapsed = (time.perf_counter() - start) * 1000

        with allure.step("记录性能指标"):
            allure.attach(
                f"耗时: {elapsed:.2f}ms\n形状: {result.shape}",
                name="性能数据",
                attachment_type=allure.attachment_type.TEXT
            )
```

```
        # 性能断言
        assert elapsed < 1000, f"性能不达标: {elapsed}ms > 1000ms"

    @allure.severity(allure.severity_level.CRITICAL)  # 知识点：设置严重级别
    def test_memory_usage(self):
        """测试内存使用"""
        import tracemalloc

        tracemalloc.start()

        op = MatMulOperator()
        A = np.random.randn(1000, 1000).astype(np.float32)
        B = np.random.randn(1000, 1000).astype(np.float32)

        _ = op.forward(A, B)

        current, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        allure.attach(
            f"当前内存: {current / 1024 / 1024:.2f}MB\n峰值内存: {peak / 1024 /
1024:.2f}MB",
            name="内存使用",
            attachment_type=allure.attachment_type.TEXT
        )
```

12. pytest.ini (配置文件)

```
[pytest]
# 测试路径
testpaths = tests

# 输出配置
addopts =
    -v                          # 详细输出
    -s                          # 不捕获print
    --tb=short                  # 简洁的traceback
    --strict-markers            # 知识点：严格标记模式，未注册的mark报错
    --alluredir=./allure-results  # Allure结果目录
    -n auto                     # 知识点：pytest-xdist并行执行
    --reruns 2                  # 知识点：pytest-rerunfailures失败重跑
    --reruns-delay 1            # 重跑延迟

# 自定义marker注册
markers =
```

```
    smoke: 冒烟测试
    slow: 慢速测试
    threading: 多线程测试
    integration: 集成测试
    benchmark: 性能基准测试

# 日志配置
log_cli = true
log_cli_level = INFO
log_cli_format = %(asctime)s [%(levelname)s] %(message)s
log_cli_date_format = %Y-%m-%d %H:%M:%S

# 最小Python版本
minversion = 3.8
```

🎯 运行测试

bash

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 运行所有测试
pytest

# 3. 只运行smoke测试
pytest -m smoke

# 4. 跳过slow测试
pytest -m "not slow"

# 5. 并行执行
pytest -n 4
```